Fase 0 — Auditoría y línea base

Rama: `fase-0-auditoria` · Ejecución: 14/10/2026 · Estado: completada (pendientes en la sección 8).

Este archivo tiene dos partes:

- **Parte 1:** informe técnico de lo auditado y verificado.
- **Parte 2:** guía de estudio de la base sobre la que se construye el TP.

Convención de rutas: `SRC` = `src/main/java/ar/edu/iua/iw3`.

Parte 1 — Informe técnico

1. Alcance y resultado

La Fase 0 **no modifica código**. Su objetivo es saber con qué se cuenta antes de implementar.

Pregunta	Respuesta
¿Compila?	Sí. `BUILD SUCCESS`, 50 fuentes, Java 21.
¿Arranca contra MySQL?	Sí, en unos 9 s.
¿Funciona la seguridad JWT?	Sí: login, token por header y por query param, rechazo de token inválido.
¿Hay dominio de gas?	No. Todo el código es el proyecto demo de la cátedra (Product, Category, cli1, cli2).
¿Se puede empezar la Fase 1?	Sí, teniendo en cuenta los hallazgos de la sección 5.

2. Archivos de la fase

Archivo	Acción	Motivo	Responsabilidad
`docs/plan-de-trabajo.md`	Creado	Plan, decisiones aprobadas y forma de trabajo	Documentación
`docs/sql/seed-dev-users.sql`	Creado	Usuario de prueba para obtener un JWT	Datos de desarrollo
`docs/fase-0-auditoria.md`	Creado	Este informe y la guía de estudio	Documentación

No se modificó ningún archivo de `src/` ni el `pom.xml`.

3. Entorno verificado

Componente	Versión o estado
Java	Temurin 21.0.12
Maven	Wrapper `mvnw`, Maven 3.9.16
Spring Boot	4.1.0 (la cátedra usa 4.1.1)
MySQL	`mysql:8.0` en Docker, contenedor `iw3-gas-mysql`, puerto 3306, base `iw3_db`
Docker	29.2.1 (el daemon estaba apagado y se inició durante la fase)
Sistema operativo	Windows 11

En esta máquina ya existía el contenedor `iw3-mysql` de la cátedra (puerto 33306, base `iw3_2026`). Como el `docker-compose.yml` del TP usa el mismo `container_name`, `docker compose up` falla con "Conflict". La línea base se levantó

con un archivo override **fuera del repo** que solo cambia el nombre a `iw3-gas-mysql`; el contenedor de la cátedra no se tocó (ver hallazgo H-4).

4. Qué existe y qué falta

Área	Estado	Evidencia
Dominio de gas (Orden, Camión, Chofer, Cliente, Producto, detalle, log de estados)	**No existe**	Ninguna clase del dominio en `SRC`
Arquitectura en capas	Existe (demo)	`SRC/model`, `SRC/model/persistence`, `SRC/business`, `SRC/controllers`
Persistencia	Existe	JPA + MySQL con `ddl-auto=update`; sin migraciones
Excepciones y respuesta estándar	Existe	`SRC/business/exception/*`, `SRC/util/StandartResponse*.java`
Seguridad JWT	Existe	`SRC/security`, `SRC/auth/**`; sin usuarios sembrados
Perfiles	Parcial	`SRC/config/profile/*ScanConfig.java`; los perfiles de Maven no activan el de Spring (H-3)
Swagger/OpenAPI	**No existe**	Sin dependencia `springdoc`
Colección Postman	**No existe**	Solo `src/test/java/ar/edu/iua/iw3/datasets/add_postman.csv` (productos demo)
Tests	Mínimo	`Iw3ApplicationTests.contextLoads`
Contenido de la clase 9 de la cátedra	No existe	Eventos, `EmailBusiness`, `Scheduler`, `getUserLogged()`, claim `internalId`
Validación de entrada	No existe	Igual que en la cátedra (sin `@Valid`)

5. Línea base ejecutada

Pruebas del 14/10/2026 contra MySQL real, con el usuario `admin` del seed.

#	Verificación	Comando o petición	Resultado
1	Compilación	`./mvnw -B -DskipTests compile`	`BUILD SUCCESS`, 50 fuentes, 15 s. Descargó `actuator`, `h2` y `java-jwt 4.4.0`, que no estaban en caché
2	Test existente	`./mvnw -B test`	1 test, 0 fallas, 14,7 s
3	Arranque	`./mvnw -B spring-boot:run`	`Started Iw3Application in 8.8 s`
4	Sin token	`GET /api/v1/products`	401 con `WWW-Authenticate: Basic`
5	Login correcto	`POST /api/v1/login` (`username`, `password`)	200 `text/plain`, JWT HS512
6	Con token	`GET /api/v1/products` + `Bearer`	200 `[ ]`
7	Token por query	`GET /api/v1/products?authtoken=<jwt>`	200
8	Token inválido o alterado	`Bearer abc.def.ghi` y firma modificada	401
9	Login fallido	Contraseña incorrecta / usuario inexistente	401; con contraseña incorrecta el cuerpo viene vacío (H-2)
10	Negocio: no encontrado	`GET /api/v1/products/999`	404 + `StandartResponse`
11	Negocio: alta	`POST /api/v1/products`	201 + `location: /api/v1/products/1`
12	Negocio: duplicado	Mismo `POST`	302 + `StandartResponse`

#	Verificación	Comando o petición	Resultado
13	Negocio: baja	`DELETE /api/v1/products/1`, luego `GET`	200, luego 404
14	Errores sin controller	JSON malformado, `Content-Type` erróneo, ruta inexistente (token válido)	401 sin cuerpo; se esperaba 400, 415 y 404 (H-1)
15	Documentación	`GET /v3/api-docs`, `GET /swagger-ui.html`	401 (no hay springdoc)
16	Perfil por Maven	`./mvnw -Pmysqldev spring-boot:run`	Log: `Perfil activo default` (H-3)
17	Perfil por Spring	`./mvnw -Dspring-boot.run.profiles=cli1 spring-boot:run`	Log: `Perfil activo cli1`; `/integration/cli1/products` responde
18	JSON externo (cli1)	`POST …/cli1/products/b2b` con distintas claves de precio	`price`: 0.0 · `price_product`: 0.0 · `product_price` decimal: 50.5 · `product_price` entero: 0.0 (H-6, H-7)

Los productos creados durante las pruebas se eliminaron.

6. Hallazgos

Cada hallazgo indica evidencia, causa, impacto y propuesta. Ninguno se corrigió en esta fase.

H-1 — Errores sin controller devuelven 401 sin cuerpo (impacto alto)

- **Evidencia:** prueba 14. El log muestra `DefaultHandlerExceptionResolver: Resolved HttpMessageNotReadableException` y `HttpMediaTypeNotSupportedException`.
- **Causa:** Spring resuelve el error (400/415) y el contenedor reenvía a `/error`; `anyRequest().authenticated()` lo bloquea porque la sesión es STATELESS.
- **Impacto:** SAP, TMS y el sistema de carga no distinguen un payload inválido de un problema de autenticación. Los casos de error de Postman serían engañosos.
- **Propuesta:** permitir `/error` en `SecurityConfiguration` (una línea). Es un desvío menor respecto de la cátedra; se aplica en la Fase 1 junto con H-2.

H-2 — Login fallido pierde el cuerpo (impacto medio)

- **Evidencia:** prueba 9. El log muestra `HttpMessageNotWritableException: No converter for [class StandartResponse]`.
- **Causa:** `AuthRestController.loginExternalOnlyToken` declara `produces = text/plain`, pero en error devuelve un `StandartResponse` (JSON).
- **Impacto:** el 401 es correcto por casualidad (vía H-1) y el mensaje se pierde. Si solo se corrige H-1, el login fallido pasaría a 500 (inferido; verificar al corregir).
- **Propuesta:** corregir H-1 y H-2 en el mismo cambio, con prueba de login fallido.

H-3 — Los perfiles de Maven no activan el perfil de Spring (impacto medio)

- **Evidencia:** pruebas 16 y 17.
- **Causa:** `application.properties` no referencia `@spring.profiles.active@` y `spring-boot:run` no propaga las propiedades de los perfiles de Maven.
- **Impacto:** el README y Postman deben usar `-Dspring-boot.run.profiles=<perfil>` o la variable `SPRING_PROFILES_ACTIVE`. Sin perfil no se aplica ningún `ScanConfig`.
- **Propuesta:** documentar el comando correcto. Al retirar cli1/cli2 (decisión 1) solo quedan `mysqldev` y `mysqlprod`.

H-4 — Colisión de nombre del contenedor MySQL (impacto medio)

- **Evidencia:** `docker compose up` falla con "container name /iw3-mysql is already in use".
- **Causa:** el compose del TP usa `container_name: iw3-mysql`, igual al contenedor de la cátedra (que además usa el puerto 33306 y otra base).
- **Impacto:** el README no se puede seguir tal cual en esta máquina.
- **Propuesta:** renombrar a `iw3-gas-mysql` en `docker-compose.yml` y en los comandos del README. Decisión pendiente (ver sección 8).

H-5 — Seguridad heredada de la cátedra (impacto medio, antes de desplegar)

- **Evidencia:** lectura de `SRC/auth/filters/AuthConstants.java`, `src/main/resources/application*.properties` y `SRC/security/SecurityConfiguration.java`.
- **Hechos:** la clave de firma del JWT está fija en el código; hay credenciales de base por defecto en `application.properties` y `application-mysqlprod.properties`; `/demo/**` es público (incluye `/demo/encodepass`); CORS acepta cualquier origen; `dev.info.enabled=true` en el perfil por defecto hace que las respuestas de error incluyan la traza completa (visible en la prueba 10).
- **Propuesta:** mover secretos a variables de entorno y cerrar `/demo/**` en la etapa de seguridad del examen final. Para el parcial, no subir credenciales reales.

H-6 — `JsonUtiles.getDouble` descarta números enteros (heredado de la cátedra)

- **Evidencia:** prueba 18: `"product_price": 50` se guardó como 0.0 y `50.5` como 50.5.
- **Causa:** el método exige `isDouble()` y el entero `50` es un `IntNode`.
- **Impacto:** un `preset` de 5000 o una masa de 1500 se leerían como 0 sin error.
- **Propuesta:** no reutilizar `JsonUtiles` para los payloads del TP; usar el *binding* directo de Jackson sobre clases de entrada. Si se reutiliza, corregir con `isNumber()`.

H-7 — Alias con espacios en `ProductCli1JsonDeserializer` (propio del fork)

- **Evidencia:** prueba 18: las claves `price` y `price_product` dieron 0.0.
- **Causa:** la cátedra usa `"a,b,c".split(",")`; el TP tiene `"a, b, c".split(",")`, por lo que los alias posteriores al primero conservan un espacio y nunca coinciden.
- **Propuesta:** desaparece al retirar el código demo (decisión 1). Si el patrón vuelve a usarse, separar con `split("\\s*,\\s*")`.

H-8 — Zona horaria (riesgo a verificar en la Fase 1)

- **Hechos:** la URL JDBC usa `serverTimezone=UTC`; la aplicación fija `America/Argentina/Buenos_Aires` en `Iw3Application.run()`, **después** de iniciar el contexto (el log muestra `Initial TimeZone` luego de `Started`).
- **Impacto posible:** desfase de 3 h en las fechas y horas de la orden.
- **Propuesta:** probar con el primer timestamp real (`OrderStatusLog`) y, si hay desfase, fijar `hibernate.jdbc.time_zone`.

H-9 a H-11 — Menores

ID	Hallazgo	Propuesta
H-9	`pom.xml` desalineado: Boot 4.1.0 vs 4.1.1, `java-jwt` 4.4.0 vs 4.6.1, `actuator` y `h2` sin uso. Compila y arranca igual.	Alinear es opcional (decisión 6 pendiente).
H-10	Falta `BaseRestController.getUserLogged()` y el resto de la clase 9. `Iw3Application` inyecta beans que no usa.	`getUserLogged()` entra en la Fase 1 (actor del log de estados). Lo demás se va con el demo.
H-11	Avisos de arranque: `HHH90000025` (dialecto explícito) y `open-in-view` activo.	Sin acción.

7. Recorrido de las peticiones

Rutas de archivo relativas a `SRC`. Los cuatro recorridos corresponden a las pruebas 5, 6, 11-12 y 14.

7.1 `POST /api/v1/login` (prueba 5)

Paso	Componente	Qué hace
1	`security/SecurityConfiguration.java` → `filterChain`	La ruta es `permitAll` solo para `POST` a `Constants.URL_LOGIN`.
2	`auth/filters/JWTAuthorizationFilter.java` → `doFilterInternal`	No hay header `Bearer` ni `authtoken`: sigue la cadena como anónimo.
3	`auth/controller/AuthRestController.java` → `loginExternalOnlyToken`	Recibe `username` y `password` (`@RequestParam`).
4	`auth/custom/CustomAuthenticationManager.java` → `authWrap`, `authenticate`	Envuelve usuario y contraseña en un `Authentication` y lo autentica.
5	`auth/UserBusiness.java` → `load` → `UserRepository.findOneByUsernameOrEmail`	`SELECT` en `users` con sus `roles` (carga EAGER).
6	`auth/User.java` → `validate`	Comprueba cuenta vencida, credenciales vencidas, bloqueo y habilitación.
7	`BCryptPasswordEncoder.matches`	Compara la contraseña con el hash guardado.
8	`AuthRestController`	Firma el JWT (HS512) con `sub`, `roles`, `email`, `version`, `exp` y responde `200 text/plain`.

7.2 `GET /api/v1/products` con token (prueba 6)

Paso	Componente	Qué hace
1	`JWTAuthorizationFilter.doFilterInternal`	Detecta `Authorization: Bearer …` (o `?authtoken=` de al menos 10 caracteres).
2	`JWTAuthorizationFilter.getAuthentication`	Verifica la firma HS512 y la expiración; arma las autoridades desde el claim `roles`.
3	`SecurityContextHolder`	Guarda el `Authentication`; si el token es inválido guarda `null` y la cadena sigue.
4	`SecurityConfiguration` (`anyRequest().authenticated()`)	Sin contexto autenticado responde 401.
5	`controllers/ProductRestController.java` → `list`	Llama a la capa de negocio.
6	`business/ProductBusiness.java` → `list` → `ProductRepository.findAll`	`SELECT` en `products`; devuelve la lista como JSON.

7.3 `POST /api/v1/products`: alta y duplicado (pruebas 11 y 12)

Paso	Componente	Qué hace
1	`ProductRestController.add`	Recibe el JSON como `Product` (`@RequestBody`).
2	`ProductBusiness.add`	Llama a `load(id)` y `load(nombre)`; si alguno encuentra, lanza `FoundException`.
3a	`ProductRepository.save`	Sin duplicado: `INSERT`; el controller responde 201 con `location`.
3b	`ProductRestController.add` (`catch FoundException`)	`StandartResponseBusiness.build(FOUND, …)` y responde 302 con JSON.

7.4 Error sin controller: JSON malformado (prueba 14)

Paso	Componente	Qué hace
1	`DispatcherServlet` (resolución de `@RequestBody`)	No puede leer el cuerpo: `HttpMessageNotReadableException`.

Paso	Componente	Qué hace
2	`DefaultHandlerExceptionResolver`	La resuelve como 400 y la registra en el log.
3	Contenedor de servlets	Reenvía internamente a `/error`.
4	`SecurityConfiguration` (`anyRequest().authenticated()`)	`/error` no tiene contexto (STATELESS): responde 401 sin cuerpo. Ver H-1.

7.5 Funciones y métodos relevantes

Son las piezas que las fases siguientes reutilizan o imitan.

`AuthRestController.loginExternalOnlyToken` — `SRC/auth/controller/AuthRestController.java`

- *Responsabilidad:* autenticar con usuario y contraseña y emitir un JWT.
- *Recibe:* `username` y `password` (`@RequestParam`) y `HttpServletRequest` (hoy sin uso: la publicación del evento LOGIN está comentada).
- *Devuelve:* `ResponseEntity<?>` con el token en texto (200) o un `StandartResponse` de error.
- *Usa:* `AuthenticationManager`, `IStandartResponseBusiness`, `AuthConstants` (`SECRET`, `EXPIRATION_TIME` = 1 h) y java-jwt.
- *Datos:* solo lectura de `users` y `roles` (indirecta). No escribe.
- *Errores:* `AuthenticationServiceException` → 500; cualquier otra `AuthenticationException` → 401. En ambos casos el cuerpo se pierde por H-2.

`CustomAuthenticationManager.authenticate` — `SRC/auth/custom/CustomAuthenticationManager.java`

- *Responsabilidad:* validar credenciales contra la base y devolver un `Authentication` con el `User` como principal.
- *Recibe:* un `Authentication` con nombre y credencial (lo arma `authWrap`).
- *Devuelve:* `UsernamePasswordAuthenticationToken(user, null, user.getAuthorities())`.
- *Validaciones:* existe el usuario; `User.validate()` (vencida, credenciales vencidas, bloqueada, deshabilitada); `PasswordEncoder.matches`.
- *Usa:* `IUserBusiness.load`, `User.validate`, BCrypt. Se registra como bean en `SecurityConfiguration.authenticationManager()`.
- *Errores:* `BadCredentialsException` (no existe o contraseña inválida), `AccountExpiredException`, `CredentialsExpiredException`, `DisabledException`, `LockedException`, `AuthenticationServiceException` (falla de base).

`JWTAuthorizationFilter.getAuthentication` — `SRC/auth/filters/JWTAuthorizationFilter.java`

- *Responsabilidad:* validar el JWT del request y construir el `Authentication`.
- *Recibe:* `HttpServletRequest` y `byHeader` (header `Bearer` o parámetro `authtoken`).
- *Devuelve:* `UsernamePasswordAuthenticationToken` con un `User` mínimo (username y email) y las autoridades del claim `roles`; `null` si el token no es válido.
- *Validaciones:* firma HS512 con `AuthConstants.SECRET` y expiración (`JWT.require(...).verify`).
- *Datos:* no consulta la base (es stateless).
- *Errores:* cualquier excepción se registra con `log.error` y devuelve `null`: token vencido y firma inválida son indistinguibles para el cliente.
- *Cuidado:* el `User` del principal no trae `roles` cargados; las autoridades viven en el `Authentication`, no en `User.getAuthorities()`.

`ProductBusiness.add` — `SRC/business/ProductBusiness.java`

- *Responsabilidad:* alta con control de duplicados por id y por nombre.
- *Recibe:* un `Product`. *Devuelve:* el `Product` persistido, ya con su id.

- *Validaciones:* ni `load(id)` ni `load(nombre)` deben encontrar un registro; con `id` nulo `load` lanza `NotFoundException`, que `add` descarta a propósito.
- *Datos:* dos lecturas en `products` y un `INSERT`.
- *Errores:* `FoundException` (duplicado) y `BusinessException` (falla de base, registrada en el log).
- *Importancia:* es el patrón que se replica para "la orden ya existe" en la Fase 1.

`StandartResponseBusiness.build` — `SRC/util/StandartResponseBusiness.java`

- *Responsabilidad:* armar el cuerpo de error estándar.
- *Recibe:* `HttpStatus`, `Throwable` y `String message`. *Devuelve:* `StandartResponse` (JSON con `message`, `code` y `devInfo`).
- *Validaciones:* ninguna; si `message` es nulo se usa `ex.getMessage()`.
- *Usa:* la propiedad `dev.info.enabled`, que decide si `devInfo` lleva la traza completa.
- *Lo llama:* el `catch` de cada controller.

8. Pendientes y criterios de aceptación

8.1 Lo que no se verificó

Elemento	Motivo
Perfil `mysqlprod` y despliegue como WAR en Tomcat	El host `mysql` y el compose de producción no están en el repo
Zona horaria con fechas reales	Todavía no hay entidades con timestamp (H-8)
Envío de mails y tareas programadas	No existen en el TP (clase 9 sin incorporar)
Tests con base H2	Ningún perfil de test la configura; `contextLoads` usa MySQL

8.2 Decisiones pendientes

> **Actualización (14/10/2026):** las decisiones 6, 10, 11, 12 y 13 ya están resueltas; ver `docs/plan-de-trabajo.md`. Esta tabla conserva el estado al cierre de la Fase 0.

#	Tema	Recomendación	Cuándo
6	Alinear `pom.xml` (Boot 4.1.1, `java-jwt` 4.6.1; retirar `actuator` y `h2`)	Opcional: con el `pom.xml` actual compila y arranca	Antes de la Fase 1
10	Renombrar el contenedor a `iw3-gas-mysql` (H-4)	Sí	Fase 1
11	Corregir H-1 y H-2 en un mismo cambio	Sí	Fase 1
12	Incorporar `getUserLogged()` en `BaseRestController` (H-10)	Sí	Fase 1
13	Integrar `fase-0-auditoria` a `main` antes de abrir la rama de la Fase 1	A definir	Antes de la Fase 1

8.3 Criterios de aceptación

Criterio	Estado
Auditoría del backend actual	Cumplido (secciones 4 y 5)
Entorno y dependencias verificados	Cumplido (sección 3)
Configuración de la base de datos	Cumplido, con la salvedad de H-4
Componentes existentes y faltantes identificados	Cumplido (sección 4)
Riesgos, bloqueos y decisiones identificados	Cumplido (secciones 6 y 8.2)
Línea base para detectar regresiones	Cumplido (sección 5, 18 verificaciones)

Criterio	Estado
No modificar innecesariamente el proyecto	Cumplido: sin cambios en `src/` ni en `pom.xml`

8.4 Repetir la línea base

```
docker compose up -d                        # ver H-4 si el nombre del contenedor choca
./mvnw -B test                              # contextLoads contra MySQL
./mvnw -B spring-boot:run                   # arranca en http://localhost:8080
docker exec -i <contenedor> mysql -uroot -proot iw3_db < docs/sql/seed-dev-users.sql
curl -X POST http://localhost:8080/api/v1/login -d "username=admin&password=admin123"
curl -H "Authorization: Bearer <token>" http://localhost:8080/api/v1/products
```

Si el contenedor `iw3-mysql` de la cátedra está en el mismo equipo, `docker compose up` falla (H-4). Alternativa que no toca el repo: crear un `override.yml` y levantar con `docker compose -f docker-compose.yml -f override.yml up -d`.

```
services:
  mysqldb:
    container_name: iw3-gas-mysql
```

Parte 2 — Guía de estudio de la base del proyecto

A. Objetivo de la fase

La Fase 0 no agrega funcionalidad. Resuelve un problema previo: antes de construir el circuito de órdenes hay que saber **qué hay, qué funciona y qué se rompe**.

Dentro del circuito de órdenes de carga, la base heredada aporta tres cosas que todas las fases usan:

1. La API REST en capas (controller, negocio, repositorio).
2. La autenticación JWT que protege los endpoints que consumen SAP, la balanza (TMS) y el sistema de carga.
3. El formato estándar de error (`StandartResponse`).

Sin entender esta base no se puede defender cómo se construye el resto.

B. Conceptos teóricos involucrados

Las clases y los ejemplos salen del repo de la cátedra y de la guía de clases (resumen personal, no oficial).

Concepto	Clase	Dónde se ve en el TP
Maven, `pom.xml` , Spring Boot, `@SpringBootApplication` , IoC	1 (18/8)	`pom.xml` , `SRC/Iw3Application.java`
JPA, `@Entity` , interfaz + implementación, `@Autowired`	2 (25/8)	`SRC/model/Product.java` , `SRC/business/IProductBusiness.java`
REST, `@RestController` , `ResponseEntity` , excepciones de dominio	3 (1/9)	`SRC/controllers/ProductRestController.java` , `SRC/business/exception/*`
Perfiles de Spring y de Maven, `Long` vs `long`	4 (8/9)	`src/main/resources/application-*.properties` , `pom.xml`
Herencia JPA `JOINED` , perfiles con `ScanConfig`	6 (22/9)	`SRC/model/Product.java` , `SRC/config/profile/*`
Deserializadores custom, proyecciones, queries nativas	7 (29/9)	`SRC/util/JsonUtiles.java` , `SRC/model/persistence/ProductRepository.java`
Autenticación vs autorización, BCrypt, JWT, sesión STATELESS	8 (6/10)	`SRC/auth/**` , `SRC/security/SecurityConfiguration.java`

Concepto	Clase	Dónde se ve en el TP
Eventos, `@Scheduled`, `@PreAuthorize`	9 (13/10)	**No está en el TP todavía**; se usará en el examen final (alarmas y roles)

Explicación complementaria (no vista en clase):

- *Despacho de errores (`/error`) y Spring Security:* cuando un error no lo maneja un controller, el contenedor reenvía a `/error` y esa ruta también pasa por la cadena de seguridad. Es el origen de H-1.
- *`docker-compose` y `container_name`:* dos contenedores no pueden compartir nombre en el mismo host. Es el origen de H-4.

C. Arquitectura y organización del código

```
Cliente (Postman / sistema externo)
   │  HTTP + JWT
   ▼
Cadena de filtros de Spring Security   SecurityConfiguration, JWTAuthorizationFilter
   ▼
Controller (@RestController)           controllers/*, integration/*/controllers
   ▼
Business (@Service, interfaz IXxx)     business/*, auth/UserBusiness
   ▼
Repository (Spring Data JPA)           model/persistence/*, auth/*Repository
   ▼
MySQL (Hibernate, ddl-auto=update)
```

Paquete (bajo `SRC`)	Responsabilidad	Por qué está ahí
`controllers`	Traducir HTTP: recibir JSON, elegir el código de estado, armar la respuesta	Es la única capa que conoce HTTP
`business`	Reglas: duplicados, transiciones de estado, excepciones de dominio	Se puede probar y cambiar sin tocar HTTP ni SQL
`model` y `model/persistence`	Entidades y acceso a datos	Aísla el mapeo a tablas
`auth` y `security`	Usuarios, roles, JWT y reglas de acceso	Concentra la seguridad
`util`	Respuesta de error estándar y utilidades JSON	Se comparte entre controllers
`config/profile`	Qué entidades y repositorios se escanean por perfil	Permite variantes por cliente

Las fases 1 a 5 siguen este mismo esquema: las reglas de la orden irán en `business`, y los controllers solo traducirán a HTTP.

D. Explicación del código paso a paso

D.1 La cadena de seguridad — `SRC/security/SecurityConfiguration.java`

```
http.csrf(AbstractHttpConfigurer::disable);
http.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.POST, Constants.URL_LOGIN).permitAll()
    /* ... /v3/api-docs/**, /swagger-ui/**, /ui/**, /demo/** ... */
    .anyRequest().authenticated());
http.httpBasic(Customizer.withDefaults())
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
http.addFilter(new JWTAuthorizationFilter(authenticationManager()));
```

Línea	Qué hace	Si se elimina o cambia
`csrf(...disable)`	Apaga la protección CSRF, pensada para sesiones con cookies	Con CSRF activo, todo `POST` sin token CSRF respondería 403
`requestMatchers(...).permitAll()`	Lista blanca de rutas sin login	Sin `POST /login` nadie podría obtener un token
`anyRequest().authenticated()`	Todo lo demás exige autenticación (denegar por defecto)	Se vuelve a la API abierta de la clase 3
`httpBasic(...)`	Aporta el *entry point* que responde 401 con `WWW-Authenticate: Basic`	Cambiaría la forma del 401 (y posiblemente a 403)
`STATELESS`	No crea `HttpSession`; cada pedido debe traer su token	Aparecería `JSESSIONID` y estado en el servidor
`addFilter(JWTAuthorizationFilter)`	Inserta el filtro que lee el JWT	Los tokens se ignorarían y todo daría 401

D.2 El filtro JWT — `SRC/auth/filters/JWTAuthorizationFilter.java`

```
String header = req.getHeader(AuthConstants.AUTH_HEADER_NAME);     // "Authorization"
String param  = req.getParameter(AuthConstants.AUTH_PARAM_NAME);   // "authtoken"
boolean byHeader = !(header == null || !header.startsWith(AuthConstants.TOKEN_PREFIX));
boolean byParam  = !(param == null || param.trim().length() < 10);
if (!byHeader && !byParam) { chain.doFilter(req, res); return; }
UsernamePasswordAuthenticationToken authentication = getAuthentication(req, byHeader);
SecurityContextHolder.getContext().setAuthentication(authentication);
chain.doFilter(req, res);
```

- Primero decide **de dónde** sale el token. El header tiene prioridad sobre el parámetro, y el parámetro debe tener al menos 10 caracteres.
- Si no hay token, no hace nada y deja pasar: la decisión de rechazar la toma `anyRequest().authenticated()`, no este filtro.
- Si hay token, guarda el resultado de `getAuthentication` en el `SecurityContextHolder`, incluso si es `null`. Un token inválido deja el contexto vacío y el pedido sigue como anónimo.
- *Qué pasaría si se quitara el `return`:* el filtro seguiría y llamaría dos veces a `chain.doFilter`, con respuestas duplicadas.

D.3 La emisión del token — `SRC/auth/controller/AuthRestController.java`

```
String token = JWT.create().withSubject(user.getUsername())
    .withClaim("roles", new ArrayList<String>(user.getAuthoritiesStr()))
    .withClaim("email", user.getEmail())
    .withClaim("version", "1.0.0")
    .withExpiresAt(new Date(System.currentTimeMillis() + AuthConstants.EXPIRATION_TIME))
    .sign(Algorithm.HMAC512(AuthConstants.SECRET.getBytes()));
```

Elemento	Qué es	Observación
`withSubject`	Claim `sub`: quién es el usuario	El filtro lo usa como `username`
`roles`	Lista de nombres de rol (`ROLE_ADMIN`)	Se convierten en autoridades en cada pedido
`version`	Versión propia del formato del token	Dato de la cátedra, sin efecto funcional
`withExpiresAt`	Claim `exp`: ahora + 1 hora	Vencido el token hay que loguearse de nuevo
`sign(HMAC512)`	Firma simétrica con `SECRET`	Quien conozca `SECRET` puede falsificar tokens (H-5)

El payload **no está cifrado**, solo firmado: en la prueba se decodificó con Base64 y se leyó `{"sub":"admin","roles":["ROLE_ADMIN"],"email":"admin@iw3.local",...}`.

E. Trazabilidad de los datos

Pregunta	Login (`POST /api/v1/login`)	Alta de producto (`POST /api/v1/products`)
¿De dónde vienen los datos?	Formulario HTTP: `username` y `password`	Cuerpo JSON del cliente
¿Cómo llegan al backend?	`@RequestParam` en `loginExternalOnlyToken`	`@RequestBody Product` (Jackson convierte el JSON a objeto)
¿Dónde se validan?	`CustomAuthenticationManager.authenticate`: usuario, estado de cuenta y BCrypt	`ProductBusiness.add`: duplicado por id y por nombre
¿Dónde se transforman?	`User` → claims → JWT firmado	JSON → `Product` → fila de `products`
¿Dónde se almacenan?	En ningún lado: el token no se guarda (STATELESS)	Tabla `products` (`INSERT` vía `ProductRepository.save`)
¿Cómo se recuperan?	En cada pedido, el filtro lee el token del header o del parámetro	`GET /api/v1/products` y `/{id}`
¿Quién los usa después?	`JWTAuthorizationFilter` y `SecurityContextHolder`	El controller, que arma el `location` con el id generado
¿Qué recibe el cliente?	200 `text/plain` con el JWT	201 sin cuerpo y `location: /api/v1/products/{id}`

La alta de producto es el patrón de las órdenes: la Fase 1 sigue exactamente este camino con la orden y sus entidades externas.

F. Ejemplo práctico de ejecución

Los valores son de la línea base. El usuario `admin` y su contraseña son **ficticios y de uso solo local** (ver `docs/sql/seed-dev-users.sql`).

1. Login

```
POST /api/v1/login
Content-Type: application/x-www-form-urlencoded

username=admin&password=admin123
```

Validaciones: usuario existe, cuenta habilitada, `BCrypt.matches` correcto. La base solo se lee (`users` y `roles`). Respuesta:

```
200 OK   Content-Type: text/plain;charset=UTF-8
<JWT de 256 caracteres: header.payload.firma>
```

Payload decodificado: `{"sub":"admin","roles":["ROLE_ADMIN"],"email":"admin@iw3.local","version":"1.0.0","exp":<ahora + 3600 s>}`.

2. Consulta protegida

```
GET /api/v1/products
Authorization: Bearer <JWT>
```

El filtro valida la firma y la expiración; el controller responde `200` con `[ ]` (tabla vacía). Sin el header, la misma petición responde `401`.

3. Error de negocio

```
GET /api/v1/products/999
Authorization: Bearer <JWT>
```

`ProductBusiness.load` no encuentra el id y lanza `NotFoundException`; el controller responde:

```
{ "message": "No se encuentra el Producto id=999", "code": 404, "devInfo": "<traza>" }
```

`devInfo` lleva la traza solo con `dev.info.enabled=true` (H-5); en otro caso es `null`.

G. Preguntas posibles del profesor

1. ¿Qué arquitectura tiene el backend y qué responsabilidad tiene cada capa?
2. ¿Qué arranca la aplicación y por qué `Iw3Application` extiende `SpringBootServletInitializer`?
3. ¿Cómo se conecta a la base de datos y quién crea las tablas?
4. Explicá qué pasa desde que llega `POST /api/v1/login` hasta que se devuelve el token.
5. ¿Cómo valida el backend el token en cada pedido y qué ocurre si es inválido o venció?
6. ¿Dónde se define qué rutas son públicas? ¿Por qué STATELESS y CSRF desactivado?
7. ¿Cómo se guardan y se comparan las contraseñas?
8. ¿Cómo traduce el backend una excepción de negocio a un código HTTP? Dá un ejemplo.
9. ¿Por qué un duplicado responde 302 y no 409?
10. Si mando un JSON malformado con un token válido, ¿qué responde el sistema y es correcto?
11. ¿Cómo se activa un perfil de Spring y por qué `-Pmysqldev` no lo activa?
12. ¿Qué es una línea base y cómo comprobarías que un cambio no rompió lo existente?
13. ¿Para qué sirve la interfaz `IProductBusiness` si hay una sola implementación?
14. ¿Cómo se relacionan usuarios y roles en la base de datos y cómo llegan al token?

H. Respuestas modelo

1. Es una arquitectura en capas. El *controller* (`@RestController`, por ejemplo `ProductRestController`) solo traduce HTTP: recibe el JSON, llama al negocio y convierte el resultado o la excepción en un código de estado. El *business* (`IProductBusiness` y `ProductBusiness`, con `@Service`) tiene las reglas, como no permitir duplicados, y lanza excepciones propias. El *repository* (`ProductRepository extends JpaRepository`) accede a la base con Spring Data JPA, y la *entidad* (`Product`, `@Entity`) es lo que se mapea a la tabla. Cada capa solo habla con la de abajo; por eso las reglas de estado de la orden van en `OrderBusiness` y no en el controller.

2. `@SpringBootApplication`, en `Iw3Application`, junta `@Configuration`, `@EnableAutoConfiguration` y `@ComponentScan`. El `main` llama a `SpringApplication.run`, que crea el contexto de Spring (el contenedor de inversión de control) y levanta el Tomcat embebido. Extiende `SpringBootServletInitializer` para poder desplegarse como WAR en un Tomcat externo: el `pom.xml` tiene el perfil `war-build`, activo por defecto, con empaquetado `war` y Tomcat en scope `provided`. También implementa `CommandLineRunner`: su `run` fija la zona horaria y loguea el perfil activo.

3. La conexión está en `application.properties`: `spring.datasource.url` apunta al MySQL de `localhost:3306`, base `iw3_db`, con el pool de Hikari. Las tablas las crea Hibernate al arrancar porque `spring.jpa.hibernate.ddl-auto=update`: compara las entidades con el esquema y agrega lo que falta. No hay migraciones y nunca borra ni renombra columnas. Lo comprobé en la línea base: tras `contextLoads` aparecieron `users`, `roles`, `userroles`, `products` y el resto. La base corre en Docker con `docker-compose.yml`.

4. El filtro JWT no encuentra token y deja pasar como anónimo, y `filterChain` permite `POST /login`. Llega a `AuthRestController.loginExternalOnlyToken`, que recibe `username` y `password`. Llama a `CustomAuthenticationManager.authenticate`, que busca el usuario con `UserBusiness.load`

(`UserRepository.findOneByUsernameOrEmail`), valida la cuenta con `User.validate()` y compara la contraseña con BCrypt. Si todo está bien, el controller arma el JWT con `sub`, `roles`, `email`, `version` y `exp` a una hora, lo firma con HMAC512 y responde `200 text/plain`. Si falla devuelve 401, aunque el cuerpo se pierde por H-2.

5. `JWTAuthorizationFilter`, que extiende `BasicAuthenticationFilter`, lee `Authorization: Bearer …` o `?authtoken=`. `getAuthentication` verifica la firma con `AuthConstants.SECRET` y la expiración con java-jwt; si es válido arma un `UsernamePasswordAuthenticationToken` con las autoridades del claim `roles` y lo guarda en el `SecurityContextHolder`. Si no es válido devuelve `null`, lo registra en el log y el pedido sigue anónimo, así que `anyRequest().authenticated()` responde 401. No distingue vencido de firma inválida. Y como no consulta la base, un usuario deshabilitado conserva el acceso hasta que el token expira.

6. En `SecurityConfiguration.filterChain`. Son públicos `POST /api/v1/login`, `/v3/api-docs/**`, `/swagger-ui/**`, `/ui/**` y `/demo/**`; todo lo demás exige autenticación. CSRF está desactivado porque ese ataque se apoya en cookies de sesión que el navegador envía solo, y acá no hay sesión: el cliente manda el token a mano en el header. STATELESS hace que el servidor no guarde sesión y cada pedido se autentique solo con su token.

7. Con BCrypt: el bean `BCryptPasswordEncoder` está en `SecurityConfiguration`. En `users.password` se guarda solo el hash, que incluye la sal y no se puede revertir. En el login, `CustomAuthenticationManager` llama a `PasswordEncoder.matches(plano, hash)`. Para sembrar el usuario de prueba generé el hash con `GET /demo/encodepass`, un endpoint de demo de la cátedra que hay que cerrar antes de desplegar.

8. La capa de negocio lanza excepciones propias, todas *checked*: `NotFoundException`, `FoundException` y `BusinessException`. Cada método del controller las atrapa y arma la respuesta con `IStandartResponseBusiness.build(status, ex, message)`: `NotFoundException` → 404, `FoundException` → 302 y `BusinessException` → 500. Ejemplo verificado: `GET /api/v1/products/999` devuelve 404 con `{"message": "No se encuentra el Producto id=999", "code": 404, ...}`.

9. Porque es la convención de la cátedra: en todos los controllers `FoundException` se traduce a `HttpStatus.FOUND` (por ejemplo en `ProductRestController.add`). Reconozco que 302 es semánticamente una redirección y que 409 Conflict describiría mejor un duplicado. Decidimos mantener la convención para ser coherentes con el material (decisión 4) y dejarlo documentado en Swagger. Verificado: el segundo `POST` del mismo producto devuelve 302 con el JSON de error.

10. Con token válido devolvió 401 sin cuerpo, y eso no es correcto: debería ser 400. Spring sí detecta el error (`HttpMessageNotReadableException`, que `DefaultHandlerExceptionResolver` resuelve como 400), pero la respuesta se reenvía internamente a `/error` y esa ruta también pasa por la cadena de seguridad; como la sesión es STATELESS no hay contexto y `anyRequest().authenticated()` responde 401. Es el hallazgo H-1 y se corrige permitiendo `/error`, junto con H-2, en la Fase 1.

11. Un perfil de Spring se activa con la propiedad `spring.profiles.active`: por la variable `SPRING_PROFILES_ACTIVE`, por `--spring.profiles.active=` o, con el plugin, con `-Dspring-boot.run.profiles=`. `-P` activa un perfil de *Maven*, que solo cambia propiedades del build (empaquetado, scope de Tomcat), y como `application.properties` no referencia `@spring.profiles.active@`, el valor nunca llega al runtime. Lo verifiqué en el log: con `-Pmysqldev` dice `Perfil activo default` y con `-Dspring-boot.run.profiles=cli1` dice `cli1`.

12. Es el conjunto de resultados conocidos antes de cambiar nada. Acá son 18 verificaciones (sección 5): compilar, `contextLoads`, login, token inválido, 404/201/302, etc. Después de cada fase repito los comandos de la sección 8.4 y comparo. Lo que cambia tiene que estar documentado y justificado, por ejemplo H-1, que pasa de 401 a 400 a propósito; lo que cambia sin estar documentado es una regresión.

13. Para depender de un contrato y no de una clase concreta (inversión de dependencias, la D de SOLID). El controller inyecta `IProductBusiness` con `@Autowired` y Spring resuelve la implementación `ProductBusiness`. Así se puede cambiar o simular la implementación sin tocar a quien la usa. En el TP habrá `IOrderBusiness` y `OrderBusiness` con el mismo esquema.

14. `User` tiene `@ManyToMany(fetch = EAGER) Set<Role> roles` con la tabla intermedia `userroles(id_user, id_role)`, así que los roles se cargan junto con el usuario. `getAuthorities()` los convierte en `GrantedAuthority` y `getAuthoritiesStr()` en la lista de nombres que va al claim `roles` del JWT. En cada pedido el filtro reconstruye las autoridades desde ese claim, sin consultar la base.

I. Errores frecuentes y puntos difíciles

Confusión o error	Lo correcto	Dónde se ve
Creer que un 401 siempre es un problema de token	También aparece cuando falla un error interno y se reenvía a `/error`	H-1, prueba 14
Confundir 401 con 403	401: no sé quién sos; 403: sé quién sos y no tenés permiso. Hoy no hay autorización por rol, solo "estar autenticado"	`SecurityConfiguration`
Creer que el JWT está cifrado	Está firmado, no cifrado: el payload se lee con Base64. No debe llevar datos sensibles	Sección D.3
Creer que `-Pperfil` activa un perfil de Spring	Es un perfil de Maven; el de Spring se activa con `spring.profiles.active`	H-3
Creer que `ddl-auto=update` migra la base	Solo crea y agrega; no borra ni renombra columnas	`application.properties`
Esperar que un cambio de rol o de estado de cuenta rija al instante	El token ya emitido sigue valiendo hasta que vence (1 h)	`JWTAuthorizationFilter`
Olvidar que `add` descarta `NotFoundException` a propósito	Con `id` nulo `load` lanza `NotFoundException` y eso significa "no existe, se puede crear"	`ProductBusiness.add`
Dar por válido un JSON numérico con `JsonUtiles.getDouble`	Descarta enteros y devuelve el valor por defecto	H-6, prueba 18
Separar alias con `", "` y `split(",")`	Los alias quedan con espacio inicial y no coinciden	H-7
Confundir `long` con `Long` en una entidad	`Long` admite `null` para "sin id"; `long` no	Clase 4, `Product.id`

Qué conviene estudiar con más profundidad:

- El orden de la cadena de filtros de Spring Security y dónde actúa cada uno.
- Qué es un *dispatch* de error y por qué vuelve a pasar por la seguridad.
- Diferencia entre `@Autowired(required = false)` y la inyección normal (se usa en los `*Business` de integración para que existan aunque su repositorio esté excluido).
- Cómo Hibernate traduce nombres (`idUser` → `id_user`) con la estrategia de nombres por defecto.

J. Lista de comprobación para rendir

Puedo defender esta fase si, sin mirar el material, logro:

- [ ] Dibujar las capas y decir en qué paquete y archivo está cada una.
- [ ] Explicar el recorrido de `POST /api/v1/login` nombrando archivo y método de cada paso.
- [ ] Explicar qué hace cada línea de `SecurityConfiguration.filterChain` y qué pasa si se quita.
- [ ] Decodificar un JWT y decir qué contiene, qué no contiene y por qué no está cifrado.
- [ ] Explicar cómo `NotFoundException`, `FoundException` y `BusinessException` terminan en 404, 302 y 500.
- [ ] Explicar por qué un JSON malformado devuelve 401 y qué se hará para corregirlo.
- [ ] Explicar cómo se activa un perfil de Spring y por qué `-P` no alcanza.
- [ ] Repetir la línea base con los comandos de 8.4 e interpretar el resultado.
- [ ] Explicar por qué `JsonUtiles.getDouble` pierde los enteros y por qué no se reutilizará.

- [ ] Explicar qué existe y qué falta para el dominio de gas, y en qué fase se agrega cada cosa.